

October 25, 23

Taught By: Mr. Hemant Singh
Pokhariya Sir

COMPUTER ORGANISATION

UNIT - 4

Signed Number

$-N \rightarrow$ 2's complement of $+N$.

$-7 \rightarrow$ magnitude

$+7 \rightarrow$ 0111
sign

$-7 =$ 2's complement of $+7$

$=$ 2's "

$= 1000 + 1$

$= 1001$

Multiplication of 2 signed numbers:

$\rightarrow$ Booth's Algorithm

Multiplicand (Q)

$-7 \times 3 = -21$

Multiplicand (M)

$11010111 \ M = -7 =$ 2's complement (0111)

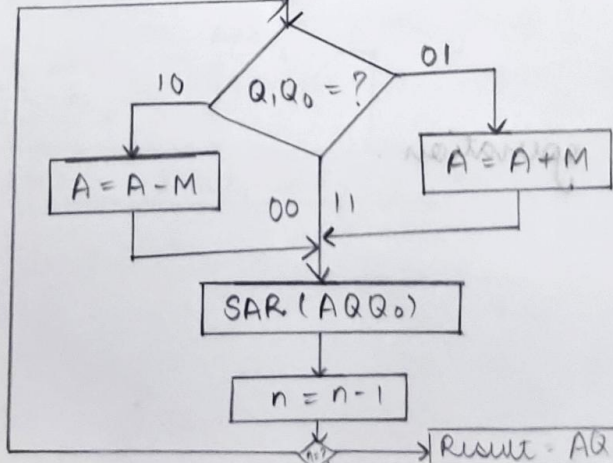
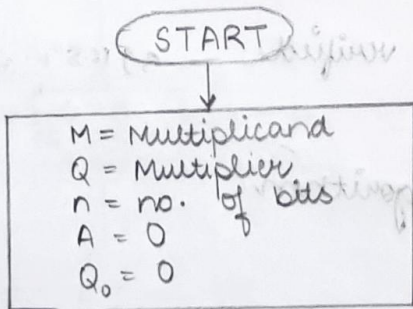
$1001 \times$

$= 1001$

$-M = -(-7) = 0111$

$10110111 \ Q_3 Q_2 Q_1 Q_0$

01101000



A $\rightarrow$ Register (Temporary)

1011 0001 0
A Q Q₀

~~10110001~~ → retains MSB

step

n	A	Q	Q ₀	operation
4	0000	0011	0	initial

MSB → 0111
retains → 0011

0111 0011 0
0011 1001 1

A = A - M = A + (-M)

SAR

0000
0111
0111

3	0001	1100	1	SAR
---	------	------	---	-----

2	1010	1100	1	A = A + M
	1101	0110	0	SAR

0001
1001
1010

1	1110	1011	0	SAR
---	------	------	---	-----

0 loop terminated

Result = AQ = 11101011

2's = 00010100 + 1 = 00010101

- 00010101

+

1

- 00010101
16 4 2

= -21. verified

3 1 6 8 4 2 1

Q Multiply -5 × 7 using booth's algorithm.

5 = 0101

-5 = 1010 + 1 = 1011

7 = 0111

n	A	Q	Q ₀	operation
---	---	---	----------------	-----------

Addition And Subtraction of two signed numbers:

→ operation

Add mag.
 $+(A+B)$

$A > B$

$A < B$

$A = B$

$$(+A) + (+B)$$

$$(+A) + (-B)$$

$$(-A) + (+B)$$

$$(-A) + (-B)$$

$$-(A+B)$$

$$+(A-B)$$

$$-(A-B)$$

$$-(B-A)$$

↳ sign comp of
 sign of A.

$$+(B-A)$$

$$(+A) - (+B)$$

$$(+A) - (-B)$$

$$(-A) - (+B)$$

$$(-A) - (-B)$$

$$+(A+B)$$

$$-(A+B)$$

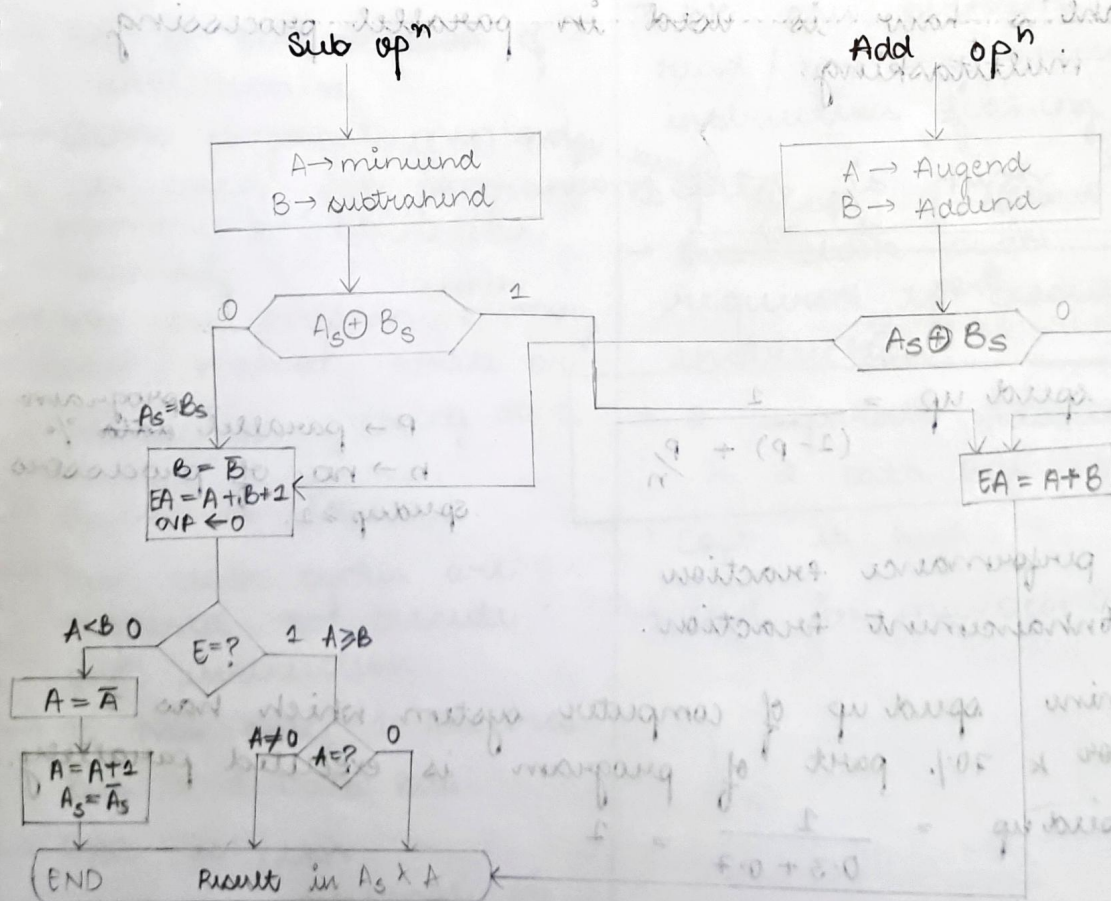
$$+(A-B)$$

$$-(B-A)$$

$$-(A-B)$$

$$+(B-A)$$

* size of both numbers should be same.



November 1, 23

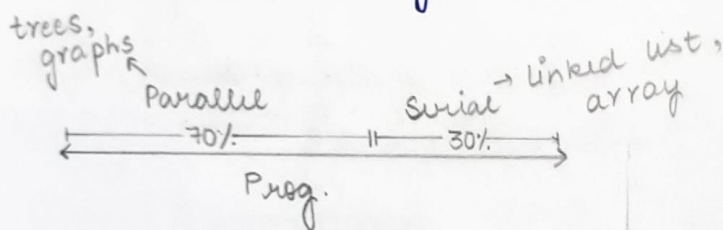
Amdahl's Law :-

→ Suppose, time taken by single processor is t_1 and by multiple processor is t_N to execute any program then,

$$\text{speed up} = \frac{t_1}{t_N}$$

* $n=1$, $T = 1 \text{ sec}$
 $n=5$, $T = 0.2 \text{ sec}$

→ Amdahl's law is used in parallel processing i.e. multitasking.



$$\text{speed up} = \frac{1}{(1-P) + \frac{P}{n}}$$

$P \rightarrow$ parallel ^{program} data %
 $n \rightarrow$ no. of processors.
 $\text{speedup} \geq 1$.

$P \rightarrow$ performance fraction
 $\rightarrow$ Enhancement fraction.

Q Determine speed up of computer system which has 1 processor & 70% part of program is executed parallelly.

→ $\text{speed up} = \frac{1}{0.3 + 0.7} = 1$

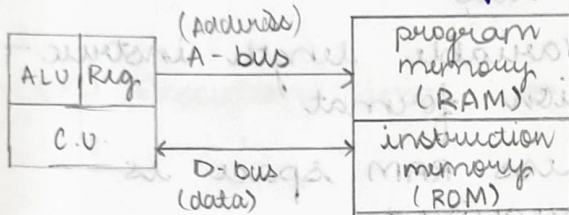
if $n=4$,

$$\text{speed up} = \frac{1}{0.3 + \frac{0.7}{4}} = \frac{4}{1.9} = 2.1052$$

VonNeumann Architecture

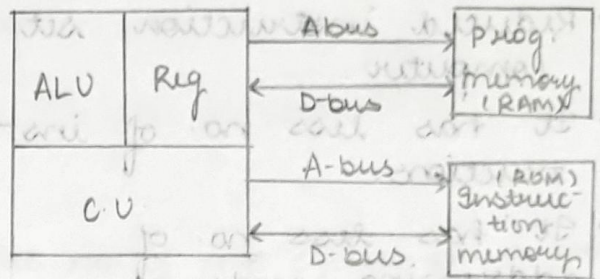
- RAM also called:
 - * Program memory
 - * Data memory
 - * Read-write memory

- ROM also known as:
 - * Code memory
 - * Instruction memory



- It is old version of architecture.
- Same physical memory is used for program memory & instruction memory.
- We can perform ^{either} memory read / memory write or instruction fetching at a time.
- Speed is low.
- Two clock cycles are required to execute one instruction.
- It has only 1 Address Bus & 1 Data Bus.
- Cost is less.
- Used in personal or small computers.

Harvard Architecture



- It is new version arch.
- Two different / separate memories are used for RAM & ROM.
- We can perform memory read / memory write and instruction fetching parallelly.
- Speed is high.
- One clock cycle is required to execute one instruction.
- 2 separate Address bus & 2 Data Bus.
- Cost is high.
- Used in microcontroller.

2nd Nov, 2023

RISC

- Reduced instruction set computer.
- It has less no. of instructions.
- It has less no. of addressing modes.
- Fixed length instruction format.
- It consumes more RAM. space is required.
- It optimizes performance by putting burden on software.
- It is highly pipelined.
- It is hardwired.
- It consumes less power.

CISC

- Complex instruction set computer.
- It has large no. of instructions.
- More no. of addressing modes.
- Variable length instruction format.
- Less RAM space is required.
- It optimizes performance by putting burden on hardware.
- It is less pipelined.
- It is micro-program.
- It consumes more power.

* $\text{Performance} = \frac{1}{\text{Execution time}}$

* $\frac{\text{Performance}^A}{\text{Performance}^B} = \frac{E.T.^B}{E.T.^A} = n$

Q Computer A runs a program in 10 seconds & computer B runs same program in 25 seconds. Which computer is faster? How much faster is A than B?

→ A is faster ~~2.5~~ 2.5 times.

Prog		m/c cycle.
ADD	→	1
SUB	→	1
STA	→	4
HLT	→	1

CPUET for a program = Total no. of m/c cycles for a prog
 $\times$ m/c cycle time.

CPU Execution Time = $\frac{\text{Total no. of m/c cycles}}{\text{clock rate}}$

Q A program runs in 10 seconds on computer A which has a 2 GHz (10^9) clock. We are trying to help a computer designer. Build a computer B which will run this program in 6 seconds. The designer has determined that a substantial increase in clock rate is possible but this increase will effect the rest of the CPU design, causing comp. B to require 1.2 times as many clock^{or machine} cycles as computer A for this program. What clock rate should we tell the designer to targate?

$$\rightarrow \text{CPU ET}^A = \frac{\text{Total no. of m/c cycles}^A}{\text{clock rate}^A}$$

$$10 \text{ sec} = \frac{\text{Total m/c cycle}^A}{2 \times 10^9 \text{ cycles/sec}}$$

$$\text{Total m/c cycles}^A = 20 \times 10^9 \text{ cycles}$$

For B,

$$6 \text{ sec} = \frac{2.4 \times 10^{10} \text{ cycles}}{\text{clock rate}}$$

$$\text{clock rate} = 4 \times 10^9 \text{ cycles/sec GHz} \\ = 4 \text{ GHz}$$

$$* \text{ Total no. of m/c cycles} = \text{Total no. of instructions} \times \text{avg. clock cycle per instruction (CPI)}$$

Q Suppose we have two implementations of the same instruction set architecture. Comp. A has a clock cycle time of 250 ps and a CPI of 2 for some program and comp. B has a clock cycle time of 500 ps and a CPI of 1.2 for the same program. Which is faster for this program & by how much?

$$\begin{aligned} \rightarrow \quad 250 &= \frac{x}{2 \times CR_A} \\ CR_A &= \frac{x}{500} \\ \text{For B,} \\ CR_B &= \frac{x}{1.2 \times 500} \\ CR_B &= \frac{x}{600} \end{aligned}$$

Now,

$$E.T._A = \frac{x}{2} \times 250 = 125x$$

$$E.T._B = \frac{x}{1.2} \times 500 = \frac{500x}{1.2}$$

$$E.T._A = 2 \times 10^8 \times 250 \text{ ps}$$

$$E.T._B = 1.2 \times 10^8 \times 500 \text{ ps}$$

$$\frac{\text{Perf}_A}{\text{Perf}_B} = \frac{E.T._B}{E.T._A} = \frac{1.2 \times 10^8 \times 500}{2 \times 10^8 \times 250} = 1.2$$

A is 1.2 times faster than B.

* CPU Time = CPU Execution Time.

$$\begin{aligned} \text{* CPU ET} &= \text{Instruction count} \times \text{CPI} \times \text{m/c cycle time} \\ &= \frac{\text{Instruction count} \times \text{CPI}}{\text{clock rate}} \end{aligned}$$

Q A compiler designer is trying to decide b/w 2 code sequences for a particular computer. The hardware designer have supplied the following:

CPI for each instrⁿ class.

	A	B	C
CPI	1	2	3

For a particular HLL statement, the compiler writer is considering two code sequences that require the following instruction count:

code sequence	A	B	C
1	2	1	2
2	4	1	1

- Which code seq. executes the most instructions?
- Which will be faster?
- Find CPI for each sequence?

i) seq 1 instruction count = $2 + 1 + 2 = 5$

seq 2 instruction count = $4 + 1 + 1 = 6$

→ seq 2 is executing more instructions.

ii) Total m/c cycles = $\sum CPI_i * C_i$

Total m/c cycles (seq 1) = $2 \times 1 + 1 \times 2 + 2 \times 3 = 10$

Total m/c cycles (seq 2) = $4 \times 1 + 1 \times 2 + 1 \times 3 = 9$

→ Code sequence 2 requires less m/c cycles and that's why faster than code seq 1.

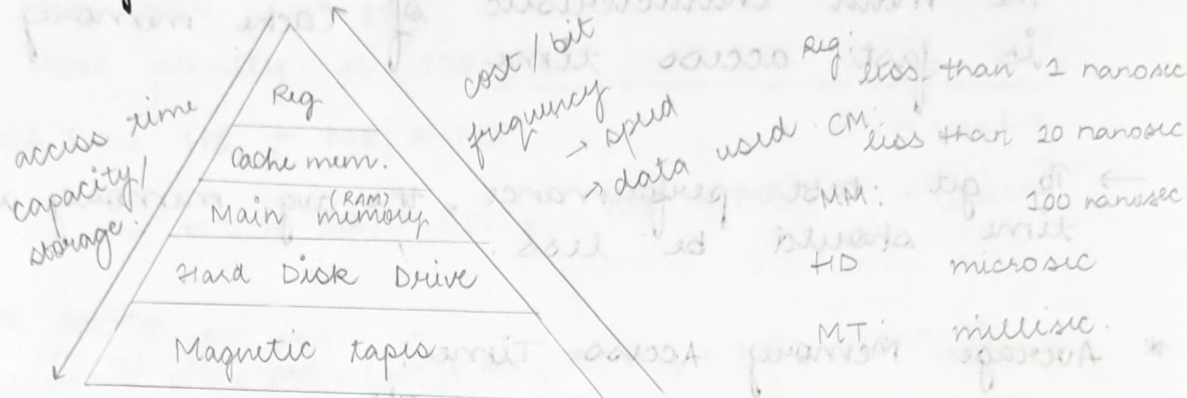
iii) $CPI^1 = 10/5 = 2$

$CPI^2 = 9/6 = 1.5$

* $Time = \frac{\text{second}}{\text{prog}}$

November 18, 23

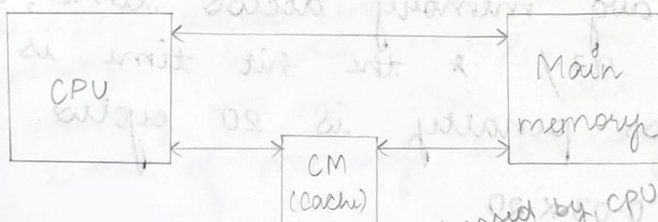
Memory hierarchy.



* RAM is made of D-RAM i.e. charging of capacitors.

* Cache is made of static RAM is using flip-flops.

Cache memory



→ CPU will search data (word → byte) first in cache memory, if found the data will be transferred to CPU & is executed. This is known as hit.

If word not found in cache, it is available in main memory & it is called miss.

$$\text{hit ratio} = \frac{\text{Total no. of hits}}{\text{Total no. of hits} + \text{Total no. of misses}} \\ (\text{Total CPU references})$$

• Ideal Hit Ratio ≥ 1 .

→ It means data is found in cache memory all times.

→ The main characteristic of Cache memory is fast access time.

→ To get best performance, the avg. memory access time should be less.

* Average Memory Access Time: time req. to copy data from MM to CM

1) $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$

2) $AMAT = \text{Hit rate} \times \text{access time of cache} + \text{Miss rate} \times \text{access time of lower memory}$

$$\text{Miss Rate (MR)} = 1 - \text{Hit Rate (HR)}$$

Q What will be avg. memory access time, if cache hit ratio is 97% & the hit time is 2 mac. cycles and miss penalty is 20 cycles.

$$\begin{aligned} \rightarrow AMAT &= 2 + 0.03 \times 20 \\ &= 2.6 \end{aligned}$$

Q If hit rate of 75% at a level of memory hierarchy & memory req. take 12 ns to complete if they hit in that level & memory req. that misses in the level takes 100 ns. Find AMAT?

$$\begin{aligned} \rightarrow AMAT &= 0.75 \times 12 + 0.25 \times 100 \\ &= 34 \text{ nanoseconds.} \end{aligned}$$

Q Assume 2 level cache system with following specifications

L1 miss hit time is 2 cycle

L1 miss rate is 2.5%

L2 hit time is 6 cycles

L2 miss rate is 17%

L2 miss penalty is 120-sec.

$$\rightarrow AMAT = HT_1 + MR_1 \times MP$$

$$= HT_1 + MR_1 \times (HT_2 + MR_2 \times MP_2)$$

if 3 level,

$$AMAT = HT_1 + MR_1 \times (HT_2 + MR_2 \times (HT_3 + MR_3 \times MP_3))$$

$$\rightarrow AMAT = 1 + 0.025 (6 + 0.17 \times 120)$$

$$= 1 + 0.025 (0.17 \times 120)$$

$$= 1.66 \text{ cycles}$$

Q Suppose the access time of cache = 5 ns. And Cache hit rate = 80%, access time of main memory = 100 ns, main memory hit rate = 99.5%, the access time of virtual memory = 10 ms. what will be the avg. access time of memory hierarchy.

ms $\rightarrow$ milli
 μ s $\rightarrow$ micro

$$\rightarrow AMAT = AT_1 \times HR_1 + (1 - HR_1) \times (AT_2 \times HR_2 + (1 - HR_2) \times MP_2)$$

$$= 5 \times 0.8 + 0.2 \times (10^7 \times 0.995 + 0.005 \times 10^6)$$

$$10^{-3} \times 10^{-6} \times 10^6$$

$$10^6 \text{ ns}$$

$$= 10024$$

$$= (5 \times 0.8) + (0.2) \times (100 \times 0.995 + 0.005 \times 10 \times 10^6)$$

$$= 10023.9 \text{ ns}$$

Mapping

→ There is transformation of word or data from main memory to cache memory.

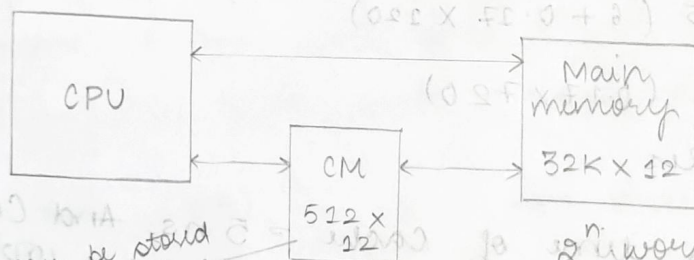
→ There are 3 mapping techniques:

i) Associative mapping.

ii) Direct mapping

iii) Set Associative mapping

* Direct Mapping



512 words can be stored with each word size being 12 bits

2^9 words

(Data is available in Octal)

$$1K = 2^{10}$$

$$1KB = 2^{10}$$

$$1MB = 2^{20}$$

$$1GB = 2^{30}$$

* Associative Mapping

$$32K = 2^n = 2^5 \times 2^{10} = 2^{15}$$

15 → 5 digits of octal

word is of 12 bits → 4 octal digits

$$2^9 = 512 = 2^9$$

9 → 3 digits of octal

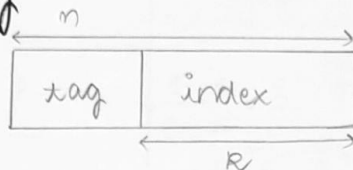
octal range →
00000 to 77777

address	data
10000	1234
21354	2312
21645	1256
70001	3124

→ If word addressed by CPU, found in cache then data is copied from CM to CPU.
Else the data is first copied from main memory to cache memory & then to CPU.

If cache memory is full, replacement algorithm is used such as FIFO.

* Direct Mapping



$n = 15$ bits
 $R = 9$ bits
index = $R = 9$ bits
tag = 6 bits

→ Divided in two parts:

MSB is tag.
LSB is index.

6 bit → 2 digits in octal
9 bit → 3 digits in octal.

tag	index	Data
00	000	1234
10	123	2345
20	456	1200
70	707	7777
77	777	

Main Memory.

Cache Memory

index	tag	data
000	25	1236
456	20	0512

* always access the lower bit (LSB) first, therefore index is accessed at first.

* Cache memory is divided into blocks.
In case of set, each block is further divided into 8 parts.

Cache Memory	Index	Tag	Data
Block 0	000	01	1224
	007	01	4567
Block 1	010	02	
	017	02	

00001

00002 → Access time ↑

Block 63
770
777

$$\frac{512}{8} = 63$$

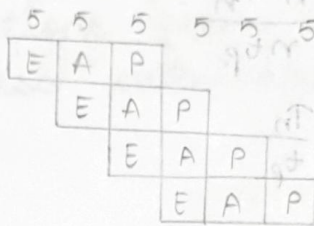
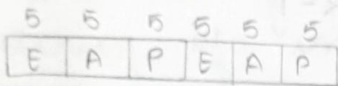
* Set Associative Mapping:

→ In this in each block there are two tags.
if two way.
four tags if four way.

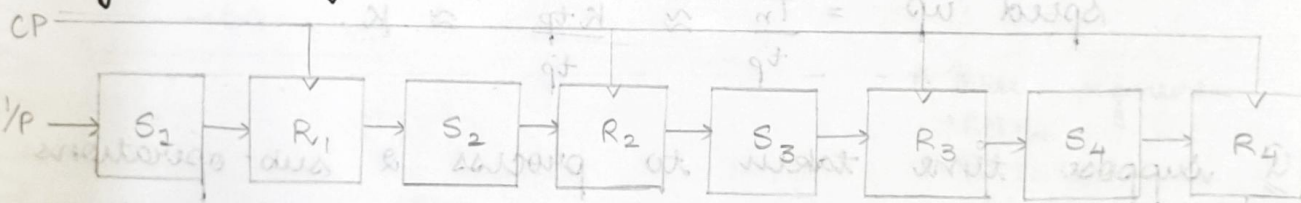
22 Nov, 22

Pipe Lining :-

→ It is a technique which decomposes a sequential process into sub-operations with each sub-operation is being executed in a specific dedicated segment and these segments work concurrently.



* Each segment is associated with 1 register.
eg:- 4 segments, 6 tasks.



→ Suppose there are 4 segments, k and no. of tasks n ,
then

$k = 4$
 $n = 6$

M/C Seq ↓	1	2	3	4	5	6	7	8	9
1	T_1	T_2	T_3	T_4	T_5	T_6			
2		T_1	T_2	T_3	T_4	T_5	T_6		
3			T_1	T_2	T_3	T_4	T_5	T_6	
4				T_1	T_2	T_3	T_4	T_5	T_6

→ To complete T_1 task, we need k clock cycles
Here $T_1 = 4 - k$.

→ Remaining 5 tasks are executed in $(n-1)$ machine cycles.
 $(n-1) = 6-1 = 5$ m/c cycles.

* suppose T_p is 1 clock cycle time.

$$\rightarrow \text{Speed up} = \frac{T_s}{T_p} = \frac{T_{\text{serially}}}{T_{\text{parallel}}} = \frac{T_{\text{without enhancement}}}{T_{\text{with enhancement}}}$$

$$\text{Speed up} = \frac{n T_n}{(k+n-1)t_p} \quad T_n \rightarrow \text{time req. to execute one task serially.}$$

$$= \frac{n T_n}{n t_p}$$

$$n \gg k-1$$

$$= \frac{T_n}{t_p}$$

$\rightarrow$ If time required in pipelining or without pipelining to complete one task is same

$$\text{Speed up} = \frac{T_n}{t_p} \approx \frac{k t_p}{t_p} \approx k.$$

Q Suppose time taken to process a sub-operations in each segment is 20 ns. Assume that there is 4 stage pipelining and there are total 100 task, then what will be speed up of that system.

$$\rightarrow k = 4, \quad n = 100$$

$$t_p = 20 \text{ ns}$$

$$\text{Speed up} = \frac{n T_n}{(k+n-1)t_p} = \frac{100 \times 4 \times 20 \text{ ns}}{(4+99)20 \text{ ns}}$$

$$= \frac{400}{103}$$

$$\approx 3.88$$

$$\approx 4.$$

Q Short note on Pipelining.

25 November, 23

Instruction cycle & pipelining:

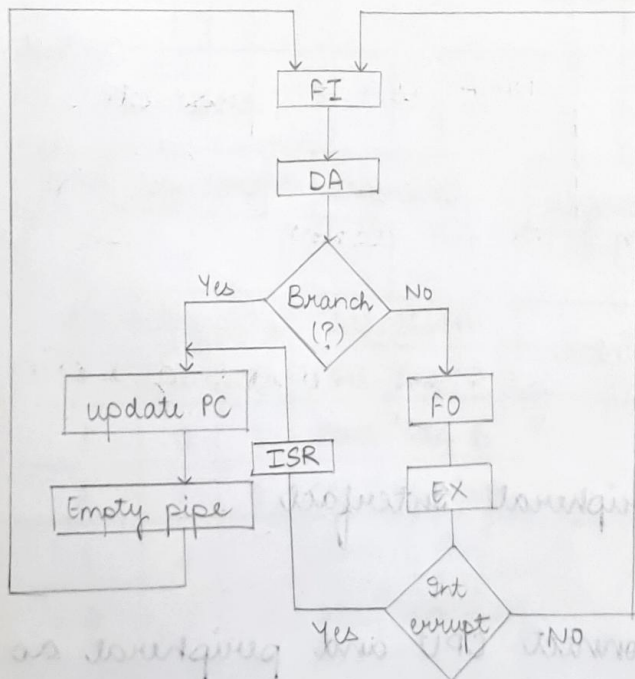
It involves following steps:

- i) Fetching of instructions from memory (FI)
- ii) Decoding and address calculation (DA)
- iii) Fetch operands (FO)
- iv) Execute (EX)
- v) stores result.

* ISR → instruction service retain.

→ FI

→ DA.



* Three segment

→ Fetch

→ Decode

→ Execute

* PC → program counter

↓
address of next instruction.

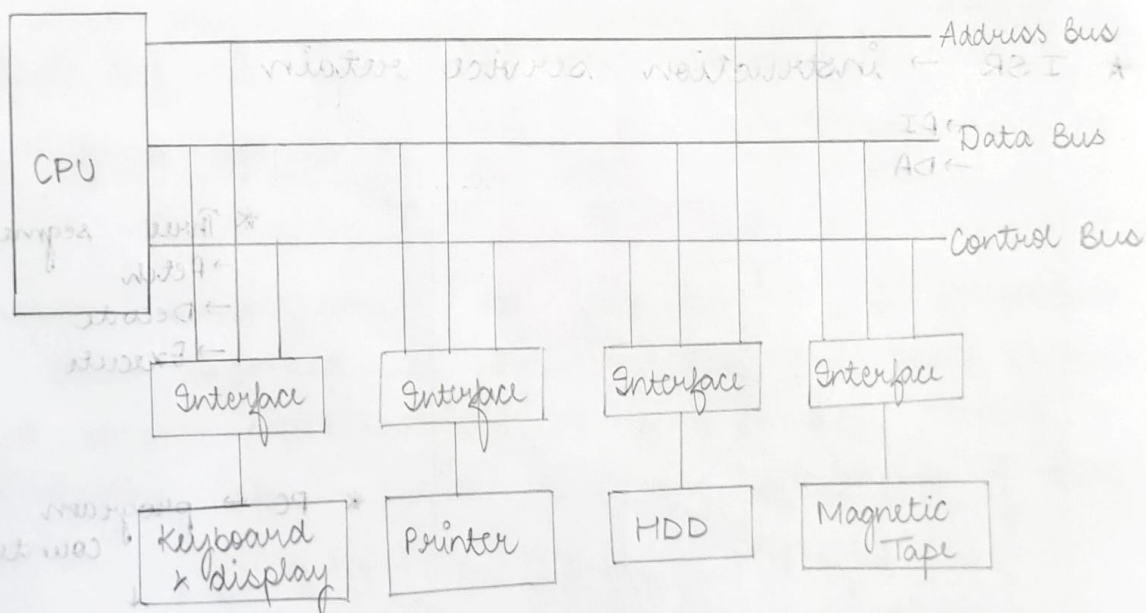
	T ₁	T ₂	T ₃	T ₄	T ₅	T ₆	T ₇	T ₈	T ₉
1	FI	DA	FO	EX					
2		FI	DA	FO	EX				
3			FI	DA	FO	EX			
4				FI	FO	FI	DA	FO	EX

FI DA FO EX

FI DA FO EX

→ after 3, there's branch : identified by decoder.
Before that, parallelly FI has occurred.

Input Output Interface

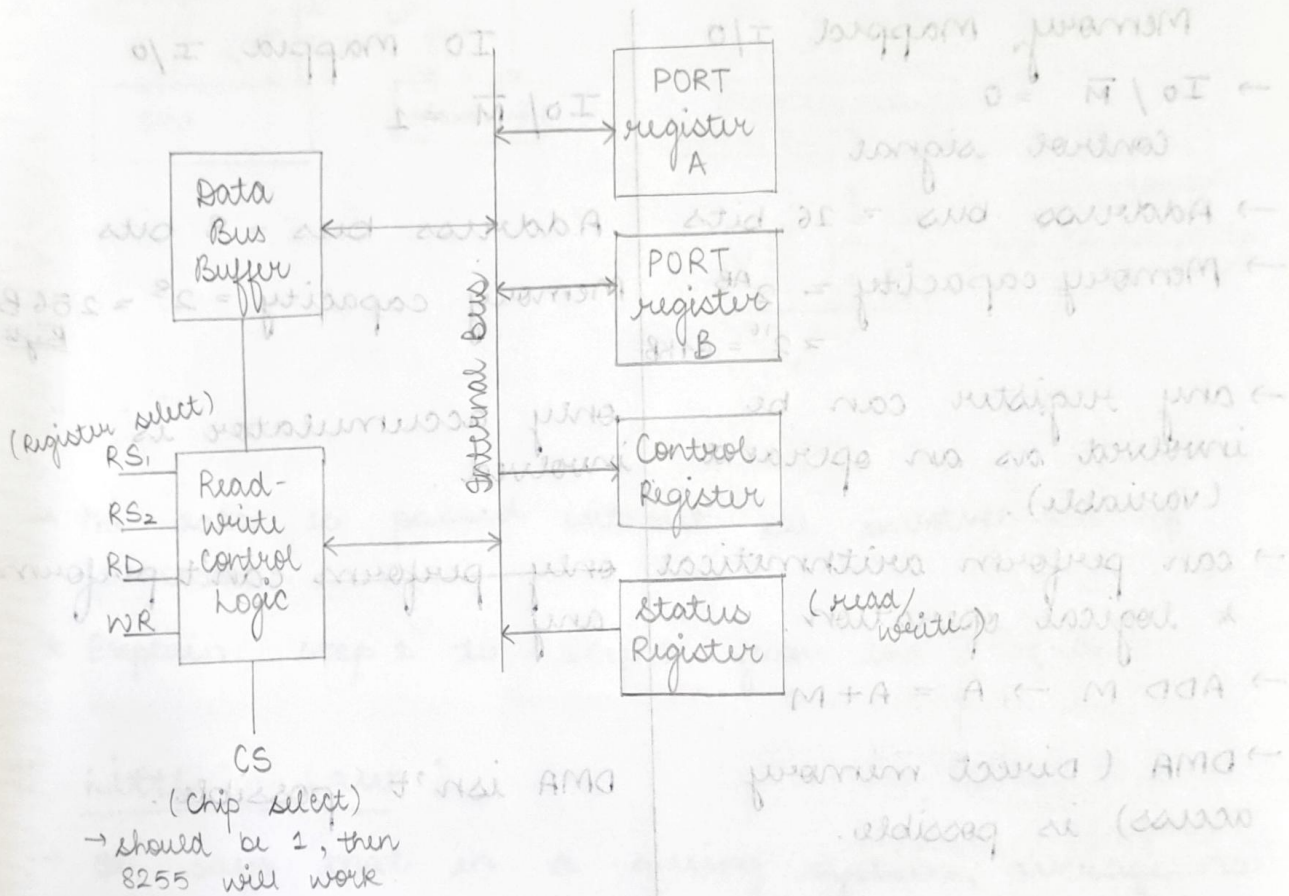


PPI → Programmable Peripheral Interface
↓
8255

→ we use interface to connect CPU and peripheral as speed of processor^{cpu} is very fast in comparison to peripheral devices, like HDD, MT, Keyboard, display etc.

→ with help of interfacing we connect CPU to different peripherals. using I/P - O/P Bus.

* Example of i/p - o/p interface:-



CS	RS_2	RS_1	Selection
1	0	0	Port Reg. A
1	0	1	Port Reg. B
1	1	0	Control Reg.
1	1	1	Status Reg.
0	X	X	Not working

* If $RD = 1$, it will perform read operation.

* If $WR = 1$, it will perform write operation.

30 Nov, 23

* 8255

(PPI) → Programmable Peripheral Interface.

Memory mapped I/O

$$\rightarrow I_0 / \overline{M} = 0$$

Control signal

→ Address bus = 16 bits

→ Memory capacity = 2^{16}
 $= 2^{16} = 64KB$

→ any register can be involved as an operand (variable).

→ can perform arithmetical & logical operation.

→ $ADD\ M \rightarrow A = A + M$

→ DMA (Direct memory access) is possible.

I/O mapped I/O

$$I_0 / \overline{M} = 1$$

Address bus = 8 bits

Memory capacity = $2^8 = 256B$
Byte

only accumulator is involved.

~~only performs~~ can't perform any.

DMA isn't possible.

DMA Transfer:

^{in most cases}
→ CPU is involved in fetching, decoding & executing.

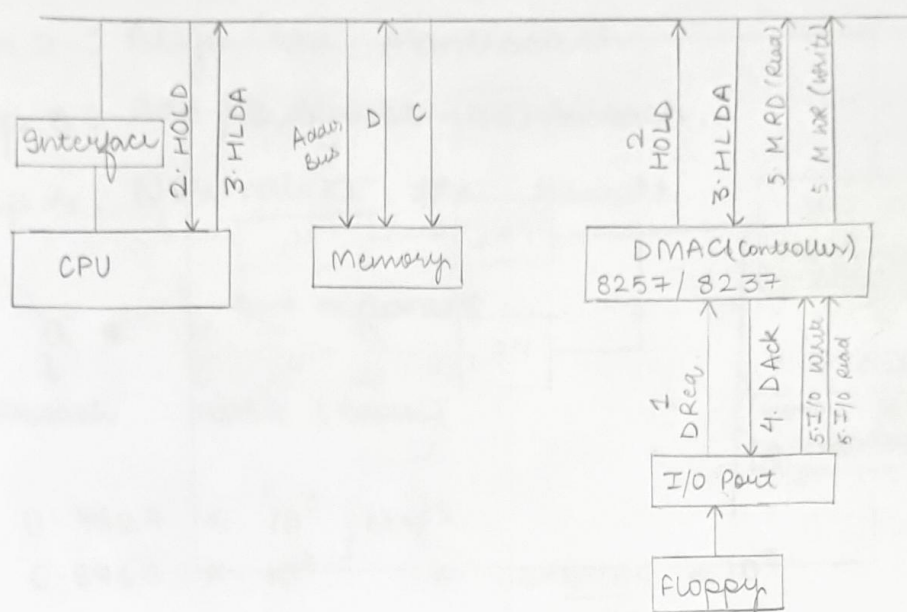
but in DMA transfer,

we can directly transfer data b/w two memories, or input / output port or interface.

There are 2 DMA control signals:

→ HOLD

→ HLDA.



Req → Request
Ack → Acknowledgment

→ the data is passed without the involvement of processor.

* Explain step 1 to step 5 from the diagram.

Little's Law :

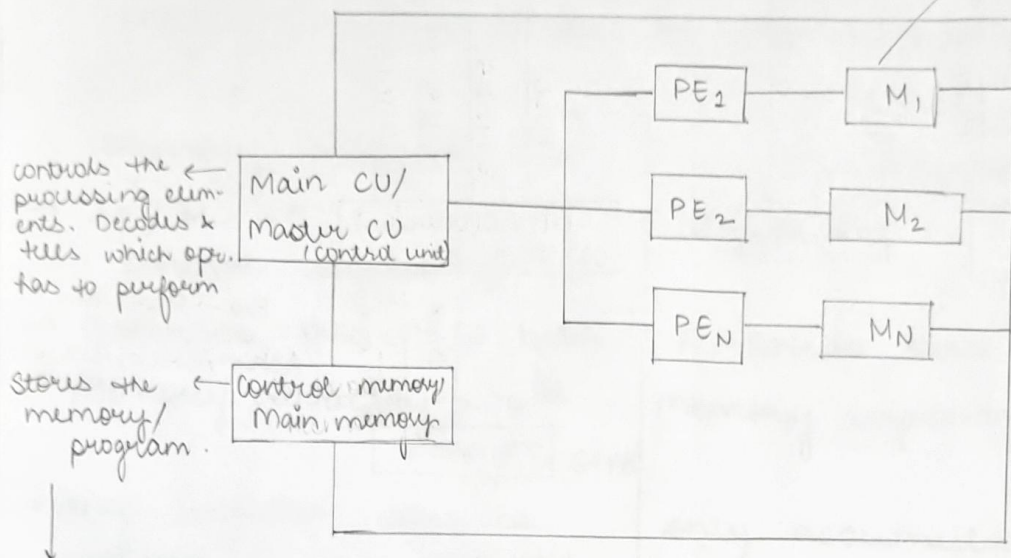
→ It says that in a queuing system, average no. of items 'L' equals the average arrival rate of items in the system, 'λ', multiplied by avg. waiting time of an item, 'w' in that system.

$$L = \lambda w$$

SIMD array processor :

→ single ~~inst~~ instruction multiple data array processor.

→ Includes register, ALU & floating point arithmetic operation



Each processing elements contains

- Register
- ALU
- floating point arithmetic hardware.

these processing elements gets ~~some~~ same instructions but execute different data.

FLYNN Taxonomy :

- SISD (single ins. single data)
- SIMD (single ins. multiple data)
- MISD (Multiple ins. single data)
- MIMD (Multiple ins. multiple data)

Floating point arithmetic operation:

Step 1: Compare the exponents.

Step 2: Align the Mantissach.

Step 3: Add / Subtract Mantissach.

Step 4: Normalize the result.

* a * x^c $\rightarrow$ exponent
↓
Mantissach radix (base)

$$\begin{array}{l} 0.9527 * 10^3 \\ 0.8460 * 10^2 \end{array} \quad \left. \begin{array}{l} \\ \end{array} \right] \text{Step 1}$$

$$= \frac{0.8460}{10} * 10^3$$

$$0.0846 * 10^3$$

$$\begin{array}{r} 0.9527 \\ 0.0846 \\ \hline \end{array} \quad \left. \begin{array}{l} \\ \end{array} \right] \text{Step 2}$$

$$1.0373 - \text{Step 3}$$

$$\text{Step 4} = 1.0373 * 10^3$$

$$= 0.10373 * 10^4 \rightarrow \text{Ans.}$$

NOTE:-

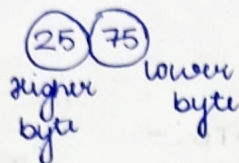
$$2.517 * 10^3$$

$$= 0.2517 * 10^4$$

↓
Mantissach

December 01, 23

Big Endian

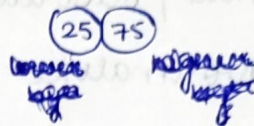


First higher byte is stored then lower byte.

25 → Address → 2000

75 → Address+1 → 2001

Little Endian



First lower byte is stored then higher byte.

75 → Address → 2000

25 → Address+1 → 2001